AI VIET NAM
@aivietnam.edu.vn

Project 2.1: Vector Database

TA. Đăng Nhã

Outline

Vector Database

1. SQL – Structure Data
2. NoSQL – Unstructure Data
3. VectorDB – Vector Data

Section 1

Distance Loss

1. L1
2. L2
3. Dot product
4. Cosine Similarity

Section 2

Project: Check Attendance

1. Project Objective
2. Search Vector Image
3. Search Vector Feature Map

Section 3

Project: Cocktail Suggestion

1. Project Objective
2. Data Processing
3. Search suitable cocktail

Section 4

Vector Database

Vector Database

❖ Data

Unstructured Data

Structured Data

Vector Database

❖ **SQL (Structured Data)**

Example: Structured Data

employees
employee_id
first_name
last_name
department_id
salary

Primary Key

n-1

Departments
department_id
department_name

Foreign Key

Vector Database

❖ Applications of SQL (Structured Data)

Web Applications

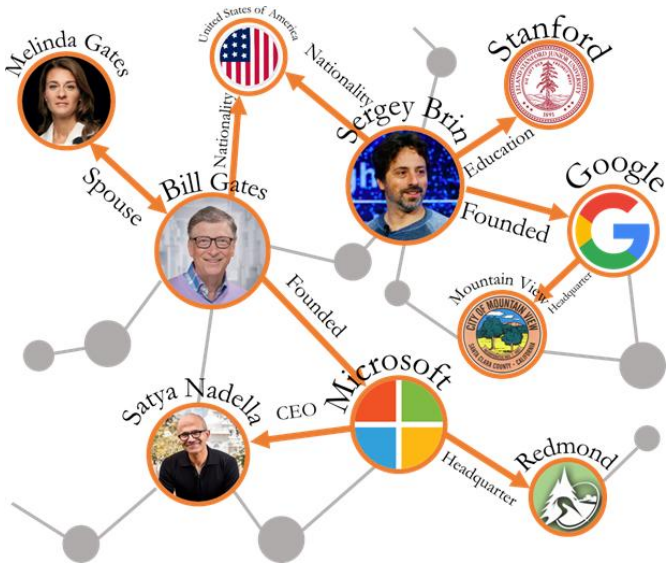
Mobile Applications

Enterprise Information Systems

Vector Database

❖ No SQL (Unstructured Data)

➢ Examples



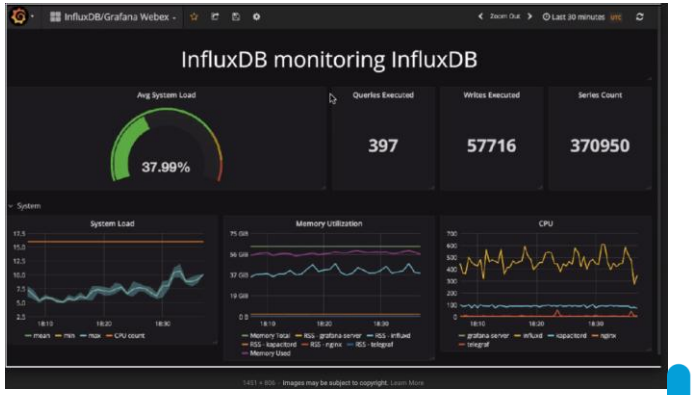

```
{JSON}
```

```json
{
  "_id": "user_001",
  "name": "Alice Nguyen",
  "email": "alice@example.com",
  "age": 28,
  "address": {
    "street": "123 Nguyen Trai",
    "city": "Ho Chi Minh City",
    "postal_code": "700000"
  },
  "hobbies": ["reading", "cycling", "photography"],
  "orders": [
    {
      "order_id": "ord_1001",
      "product": "Laptop",
      "price": 1200,
      "date": "2025-07-15"
    },
    {
      "order_id": "ord_1002",
      "product": "Headphones",
      "price": 150,
      "date": "2025-07-17"
    }
  ],
  "is_active": true,
  "created_at": "2025-01-01T12:00:00Z"
}
```

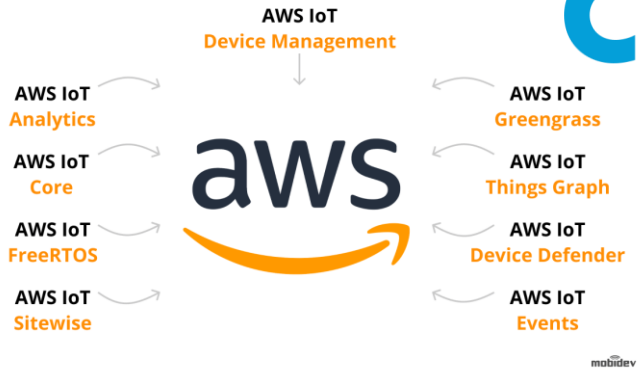
➢ Platforms

Database

❖ Applications of SQL (Structured Data)



Name	Value	Change	Chg%	Open	High	Low	Prev
INDICES							
S&P 500	4455.3	9.6	0.22%	4455.7	4462.0	4438.5	4445.7
US 100	15179.8	−20.1	−0.13%	15199.9	15253.0	15159.5	15199.9
Dow 30	34727.5	212.0	0.61%	34515.5	34731.5	34479.0	34515.5
Nikkei 225	33023.78	−218.81	−0.66%	33261.35	32267.14	32988.65	33242.59
DAX Index	15664.48	−62.64	−0.40%	15688.07	15743.90	15630.94	15727.12
UK 100	7737.0	75.5	0.99%	7661.5	7743.5	7636.0	7661.5
FUTURES							
S&P 500	4490.00	−11.50	−0.26%	4503.25	4509.50	4462.25	4501.50
Euro	1.07185	−0.00015	−0.01%	1.07330	1.07595	1.07160	1.07206



Real-time Web Applications

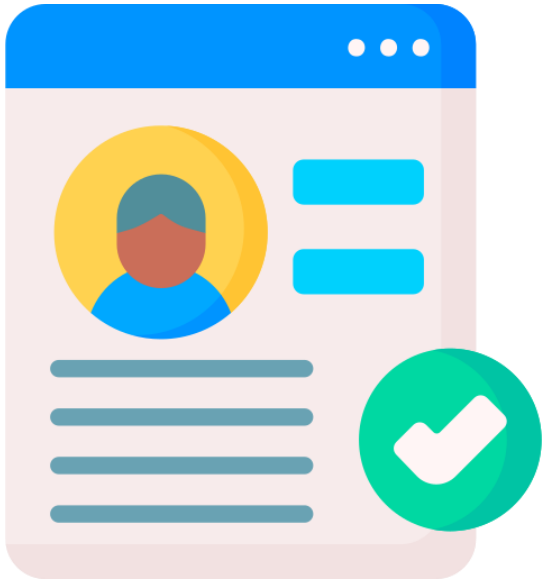
Big Data and Analytics

Vector Database

❖ Vector Database (Vector Data)

profile
user_id
first_name
last_name
avatar_link
nickname

Displaying



Storage in Server

SQL, No SQL storage purpose

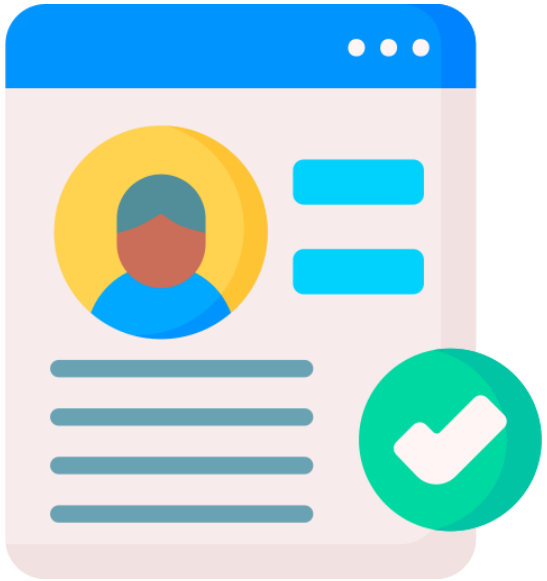
Vector Database

❖ Vector Database (Vector Data)

Storage in Server

profile
user_id
first_name
last_name
avatar_link
nickname

Displaying

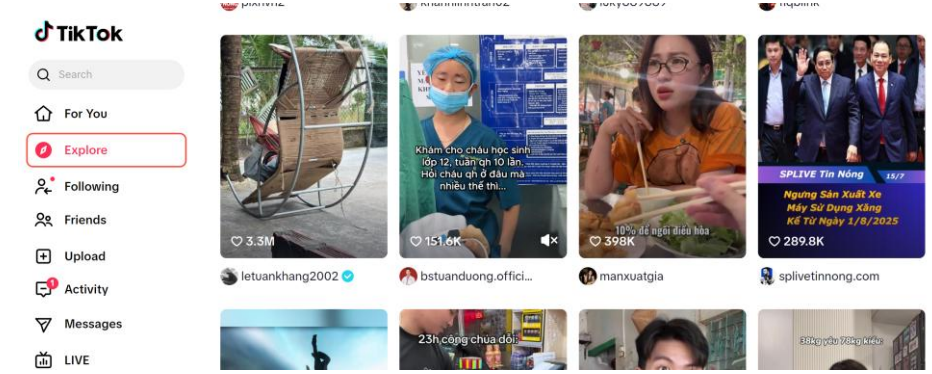


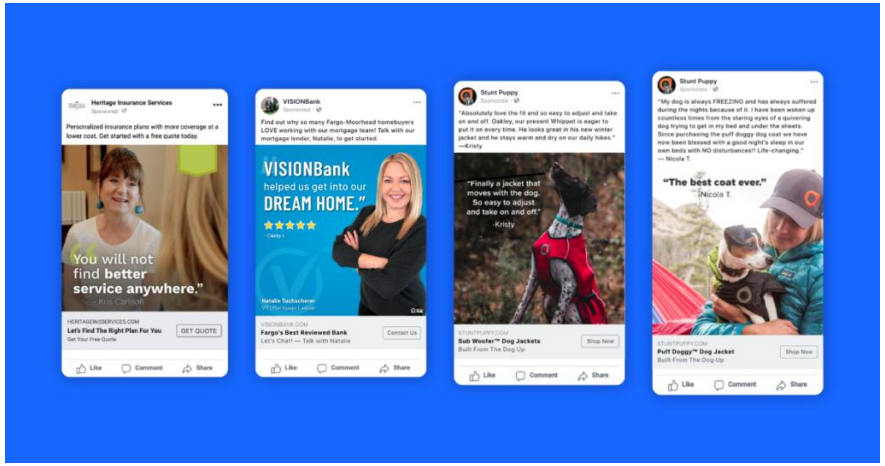
Other purpose?

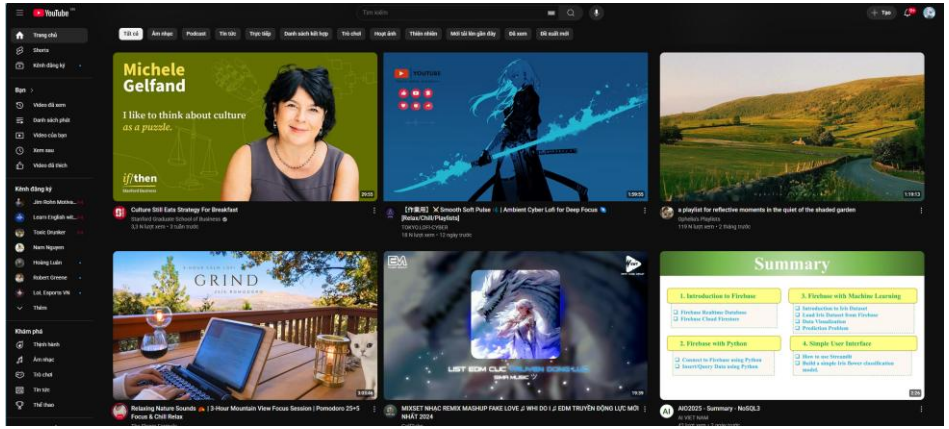
SQL, No SQL storage purpose

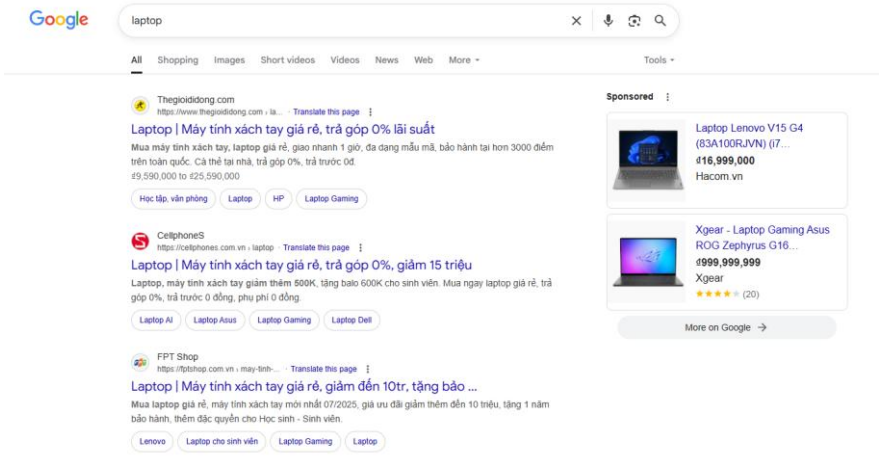
Vector Database

❖ Recommendation









Personalize Suggestion

Personalize Advertisement

Vector Database

❖ Vector Database (Vector Data)

Storage in Server

Not just displaying

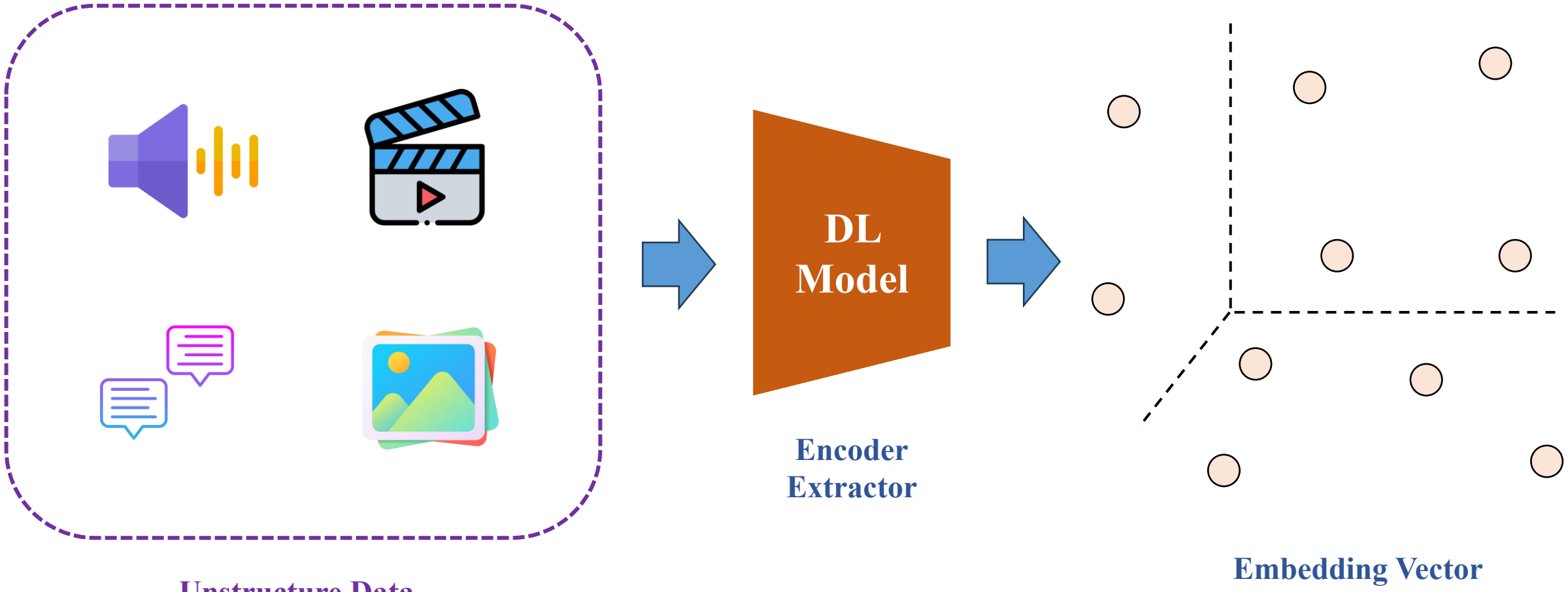
Understanding!!

Personalize Recommendation

Vector Datavase purpose

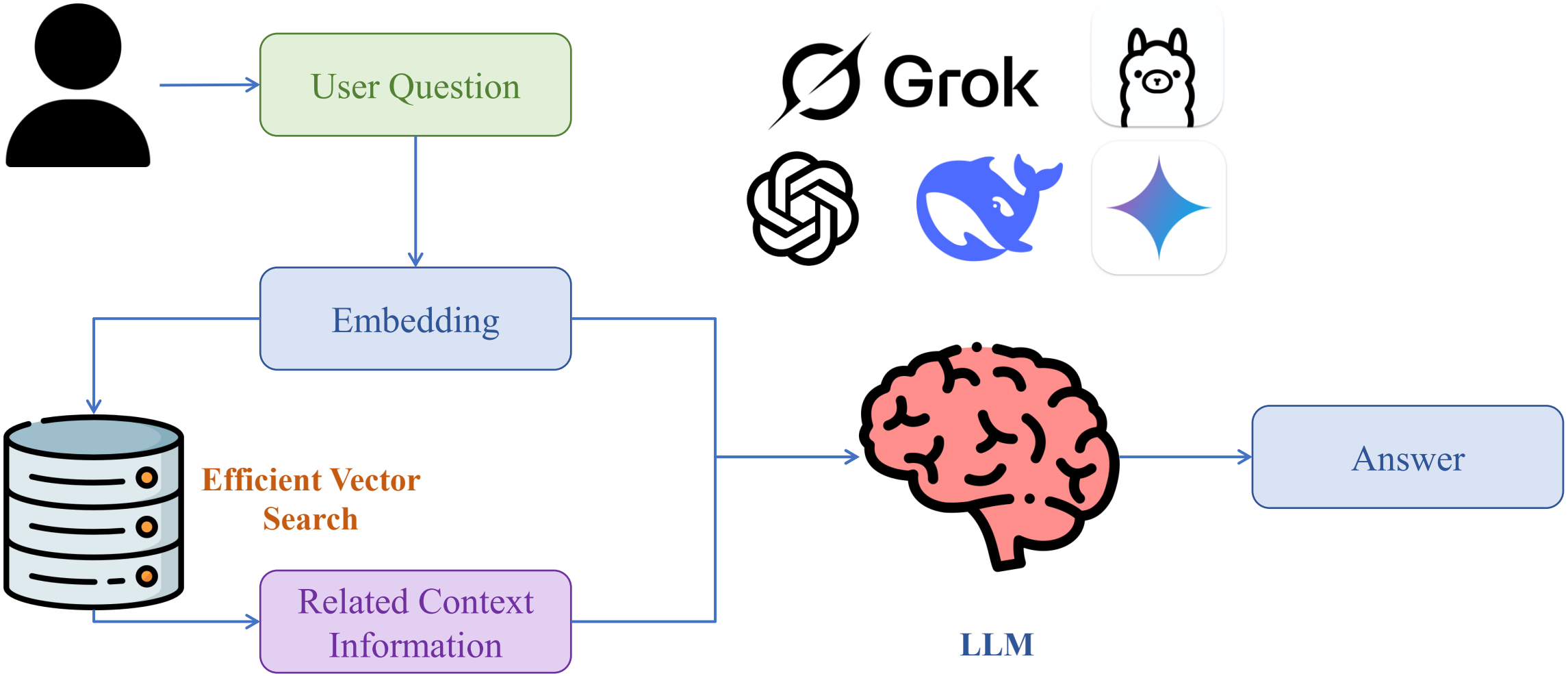
Vector Database

❖ Vector Database - Embedding



Vector Database

❖ Vector Database – Easy Search (similarity, sematic)



Distance Loss

Distance Loss

❖ How to measure disparency of two vectors

Vector A

1	1	1
1	1	1
1	1	1

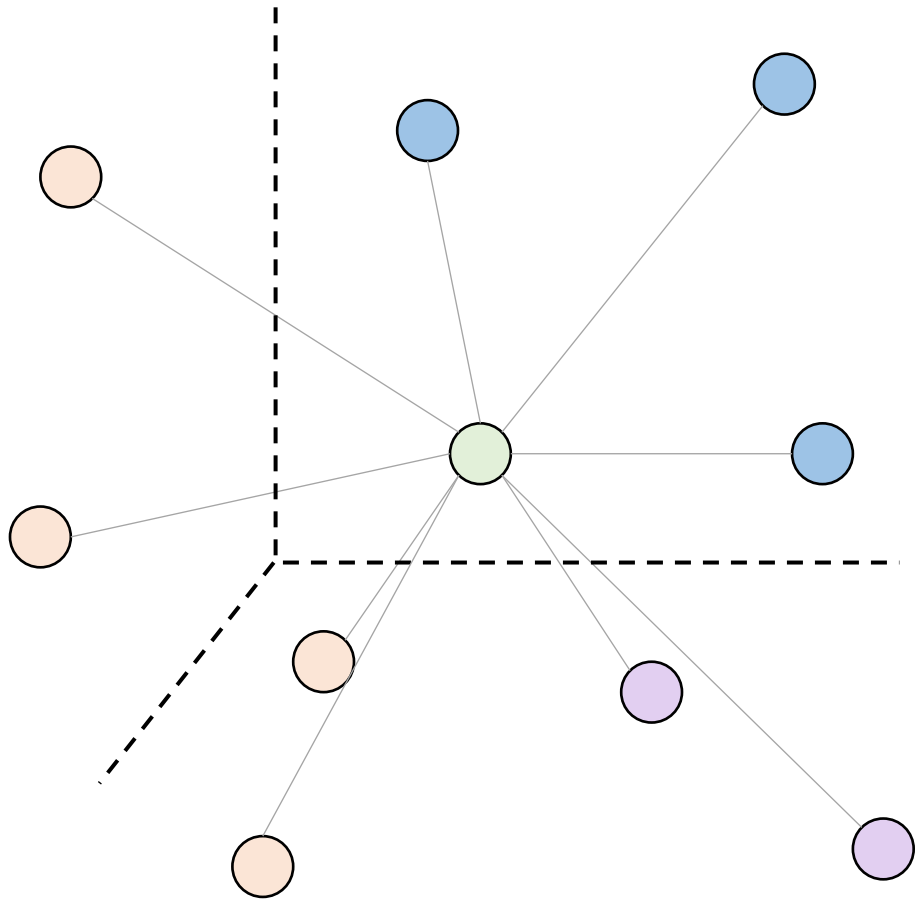
Vector B

0	0	0
0	0	0
0	0	0

Vector C

0.5	0.5	0.5
0.5	0.5	0.5
0.5	0.5	0.5

Which vector is closer to vector A or more similar to vector A?



Distance Loss

❖ L2 Distance (Euclidean Distance)

$$L2(X,Y) = \sqrt{\sum_{i=1}^{d}(x_i - y_i)^2}$$

Where:

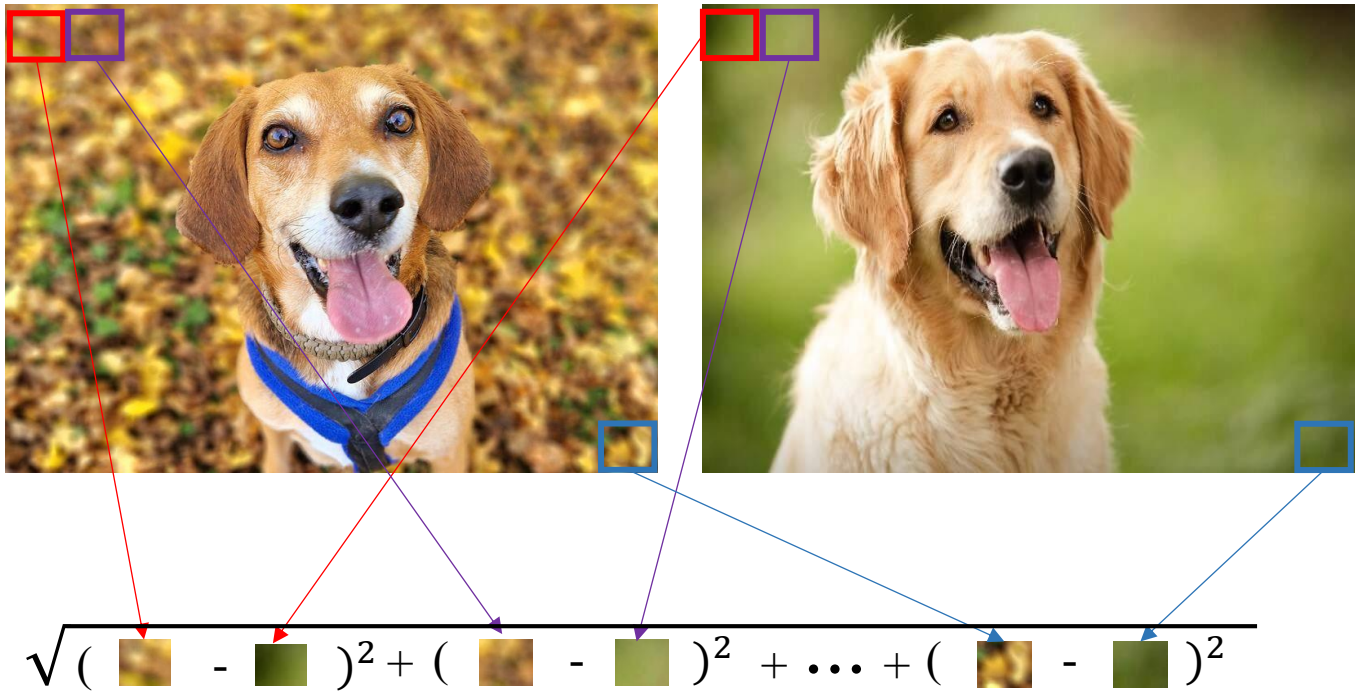
- X, Y is vector with d dimention
- X = $[x_1, x_2, x_3, \dots, x_d]$
- Y = $[y_1, y_2, y_3, \dots, y_d]$

Example 1:

X = [2, 4], Y = [4, 8]

$L2 = \sqrt{(2-4)^2+(4-8)^2} = \sqrt{20}$

Example 2:



Distance Loss

❖ L1 Distance (Manhattan Distance)

$$L1(X,Y) = \sqrt{\sum_{i=1}^{d} |x_i - y_i|}$$

Where:

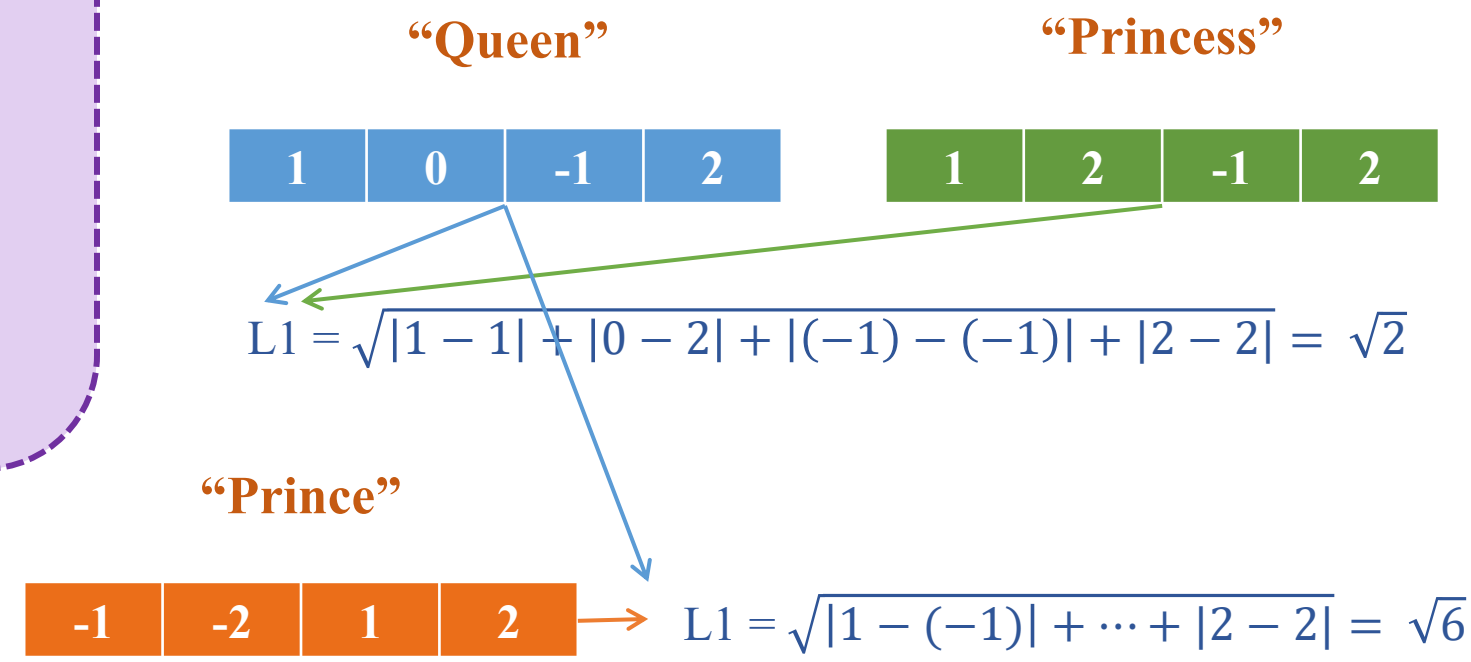
- X, Y is vector with d dimention
- X = $[x_1, x_2, x_3, \dots, x_d]$
- Y = $[y_1, y_2, y_3, \dots, y_d]$

Example 1:

X = [2, 4], Y = [4, 8]

$$L1 = \sqrt{|2-4| + |4-8|} = \sqrt{6}$$

Example 2:



Distance Loss

❖ Dot Product (Inner Product)

$$X.Y = \sum_{i=1}^{d} x_i.y_i$$

Where:

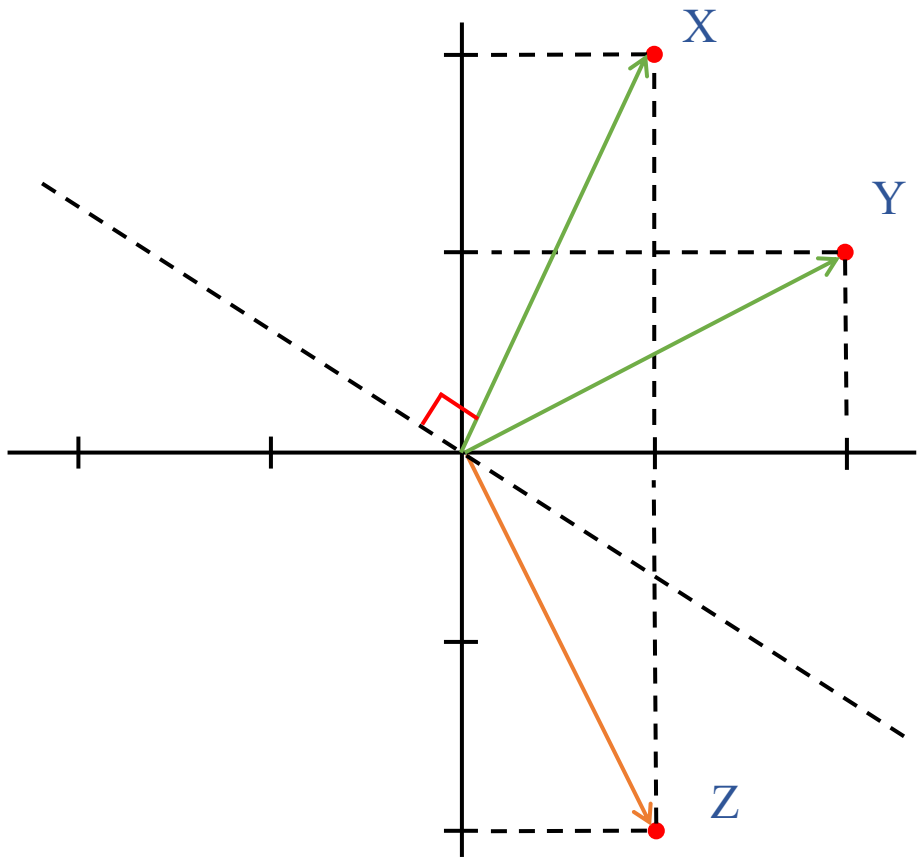
- X, Y is vector with d dimention
- X = $[x_1, x_2, x_3, \dots, x_d]$
- Y = $[y_1, y_2, y_3, \dots, y_d]$

Example:

X = [1, 2], Y = [2, 1], Z = [1, -2]

X.Y = 1.2 + 2.1 = 2 + 2 = 4

X.Z = 1.1 + 2.(-2) = 1 + (-4) = -3



Distance Loss

❖ Cosine Similarity

$$\cos(X.Y) = \frac{X.Y}{||X||.||Y||}$$

Where:

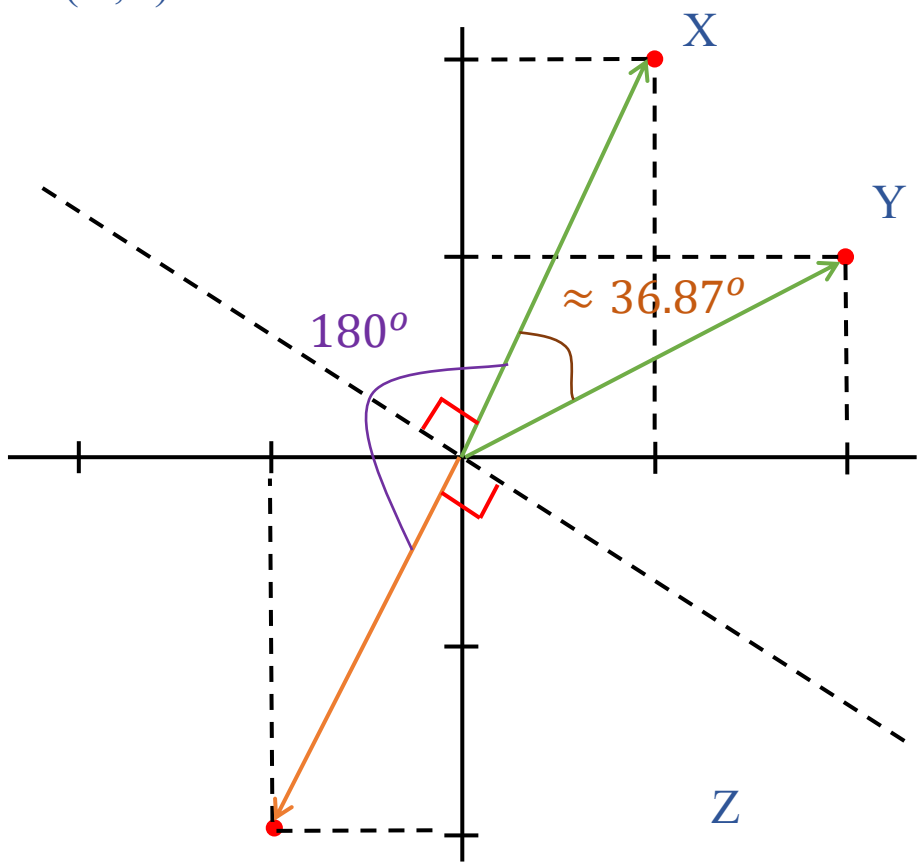
- X.Y is dot product of X & Y
- ||X|| is $\sqrt{\sum_{i=1}^{d} x_i^2}$ (Euclidean norm)

Example: X = [1, 2], Y = [2, 1], Z = [-1, -2]

||X||, ||Y||, ||Z|| = $\sqrt{5}$

cos(X,Y) = 4/5 = 0.8

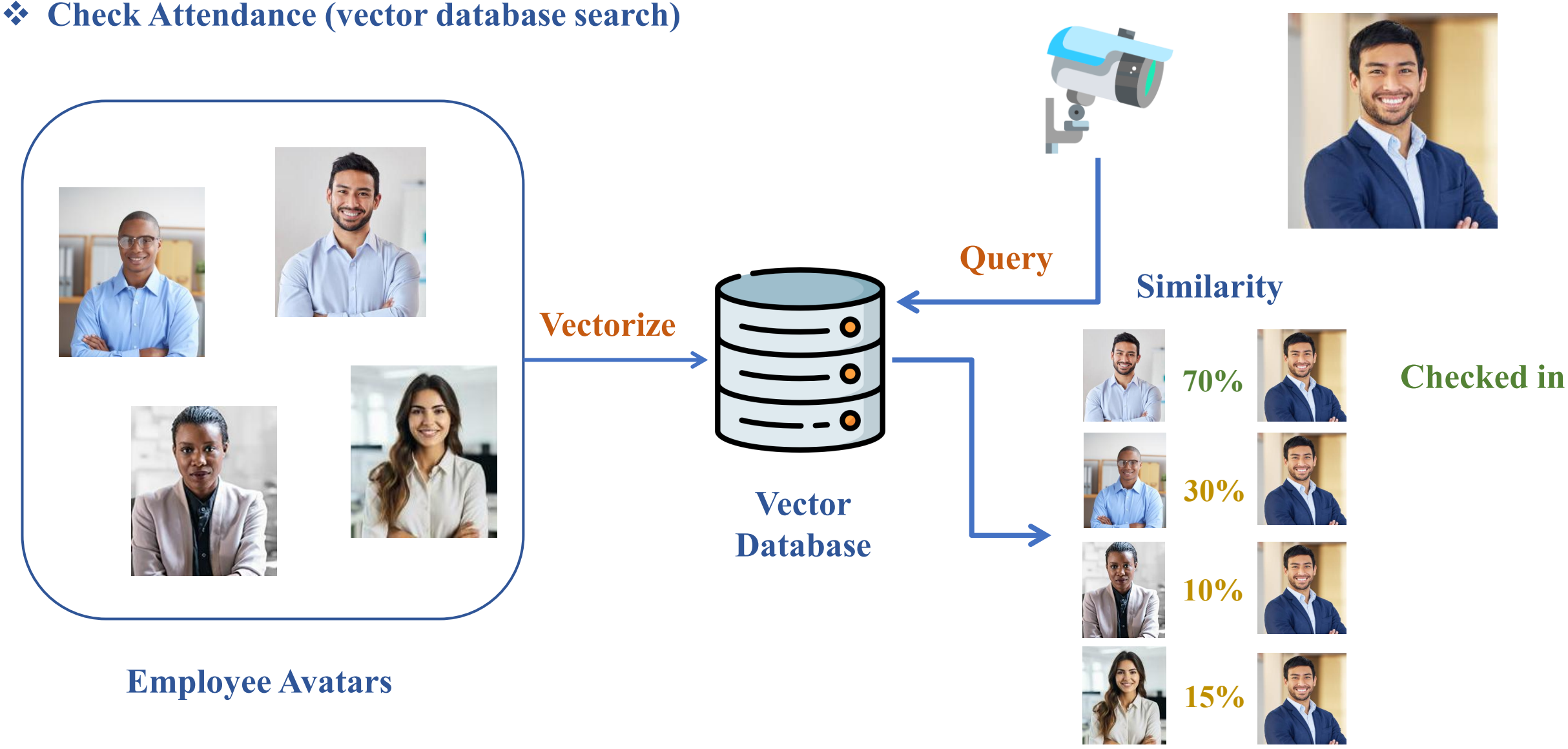
cos(X,Z) = -5/5 = -1



Project: Check Attendance

Objectives

❖ Check Attendance (vector database search)



Objectives

❖ Dataset

Dataset

```
dataset_path = 'Dataset'
```

Avatar_Dang_Nha.jpg
Avatar_Anil_Ramsook.jpg
Avatar_Quoc_Thai.jpg
Avatar_Minh_Chau.jpg
Avatar_Aaron_Guiel.jpg
Avatar_Andrew_Cuomo.jpg
Avatar_Amy_Redford.jpg
Avatar_Carla_Gay_Balingit.jpg
Avatar_Phuc_Thinh.jpg
Avatar_Anh_Khoi.jpg
Avatar_Thuan_Duong.jpg
Avatar_Hoang_Nguyen.jpg
Avatar_Thang_Duong.jpg
Avatar_Andrew_Bernard.jpg
Avatar_Tien_Huy.jpg
Avatar_Amy_Pascal.jpg
Avatar_Camille_Lewis.jpg
Avatar_Aaron_Eckhart.jpg

```
for filename in os.listdir(dataset_path):
    if filename.endswith(('.jpg', '.JPG', '.png', '.jpeg')):
        image_paths.append(os.path.join(dataset_path, filename))
        file_name = filename.split('.')[0]
        label = file_name[7:]
        labels.append(label)
```

```
image_paths = []
labels = []
```

i = 0

filename = “Avatar_Dang_Nha.jpg”

“Dataset/Avatar_Dang_Nha.jpg”

.append()

“Dang_Nha”

```
file_name = filename.split('.')[0]
label = file_name[7:]
```

Objectives

❖ Dataset

```
image_paths
```

```
['Dataset/Avatar_Dang_Nha.jpg',
 'Dataset/Avatar_Anil_Ramsook.jpg',
 'Dataset/Avatar_Quoc_Thai.jpg',
 'Dataset/Avatar_Minh_Chau.jpg',
 'Dataset/Avatar_Aaron_Guiel.jpg',
 'Dataset/Avatar_Andrew_Cuomo.jpg',
 'Dataset/Avatar_Amy_Redford.jpg',
 'Dataset/Avatar_Carla_Gay_Balingit.jpg',
 'Dataset/Avatar_Phuc_Thinh.jpg',
 'Dataset/Avatar_Anh_Khoi.jpg',
 'Dataset/Avatar_Thuan_Duong.jpg',
 'Dataset/Avatar_Hoang_Nguyen.jpg',
 'Dataset/Avatar_Thang_Duong.jpg',
 'Dataset/Avatar_Andrew_Bernard.jpg',
 'Dataset/Avatar_Tien_Huy.jpg',
 'Dataset/Avatar_Amy_Pascal.jpg',
 'Dataset/Avatar_Camille_Lewis.jpg',
 'Dataset/Avatar_Aaron_Eckhart.jpg']
```

```
from PIL import Image
avt = Image.open(img_path)
```

“Thang_Duong”

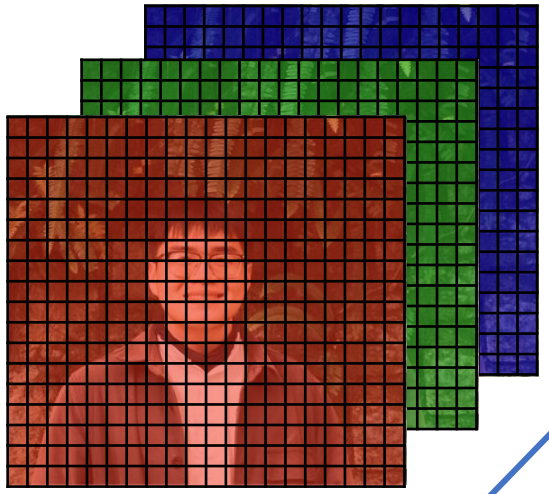
```
labels
```

[12]

```
['Dang_Nha',
 'Anil_Ramsook',
 'Quoc_Thai',
 'Minh_Chau',
 'Aaron_Guiel',
 'Andrew_Cuomo',
 'Amy_Redford',
 'Carla_Gay_Balingit',
 'Phuc_Thinh',
 'Anh_Khoi',
 'Thuan_Duong',
 'Hoang_Nguyen',
 'Thang_Duong',
 'Andrew_Bernard',
 'Tien_Huy',
 'Amy_Pascal',
 'Camille_Lewis',
 'Aaron_Eckhart']
```

Search Vector Image

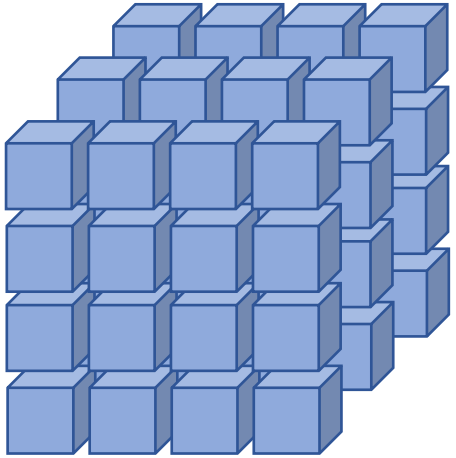
❖ **Store Image like vector**



300 x 300 x 3

3 Channels RGB

To Numpy array



3D tensor

Flatten

1D vector

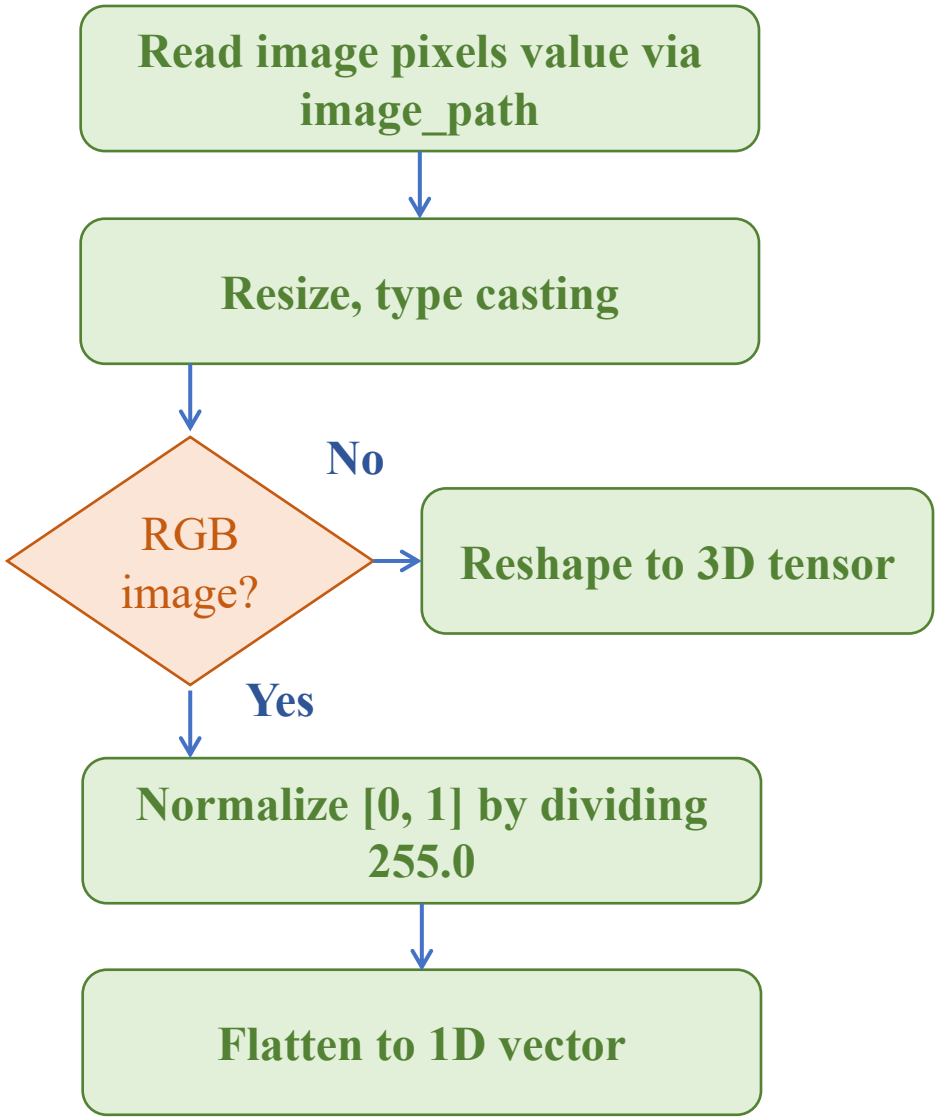
Corlor Image is 3D tensor

Why flatten?

Search Vector Image

❖ Detail Implement

```python
def image_to_vector(image_path):
    """Convert image to normalized vector"""
    img = Image.open(image_path).resize((IMAGE_SIZE, IMAGE_SIZE))
    img_array = np.array(img)

    # Handle grayscale images (convert to RGB)
    if len(img_array.shape) == 2:
        img_array = np.stack((img_array,)*3, axis=-1)

    # Normalize pixel values to [0, 1]
    vector = img_array.astype('float32') / 255.0
    return vector.flatten()
```



Search Vector Image

❖ Faiss Vector Database

"Faiss contains several methods for similarity search. It assumes that the **instances are represented as vectors and are identified by an integer**, and that the vectors can be compared with L2 (Euclidean) distances or dot products."

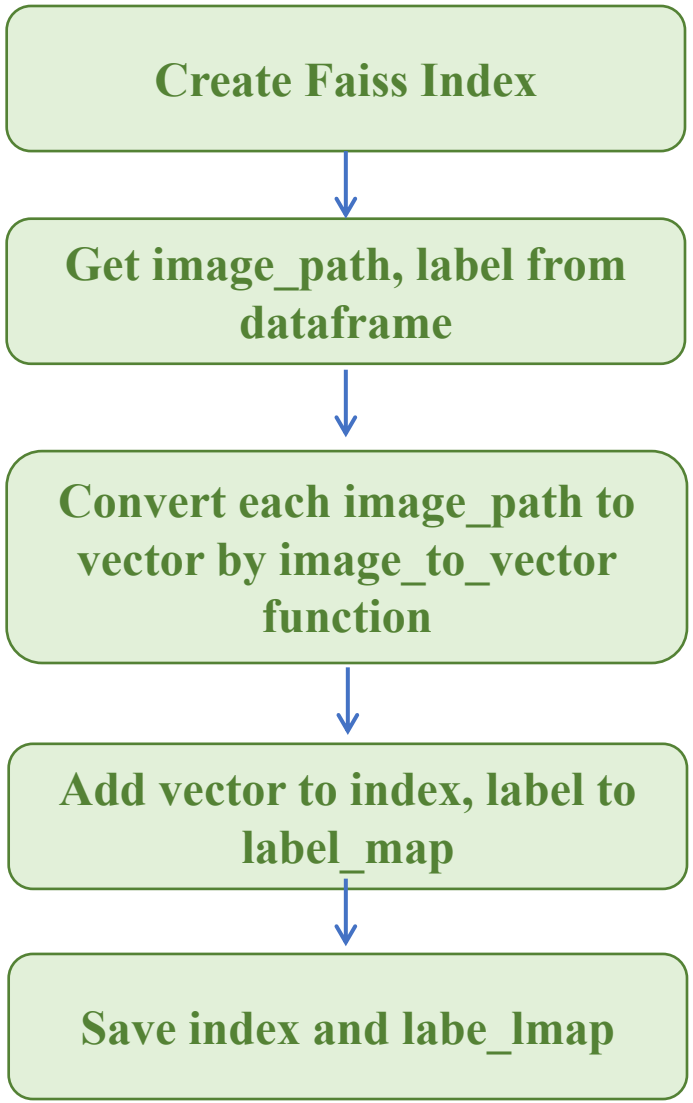
```
import faiss
# L2
index = faiss.IndexFlatL2(VECTOR_DIM)
# Dot Product
index = faiss.IndexFlatIP(VECTOR_DIM)
```

- **My vector dimension = VECTOR_DIM**
- **I use dot product to measure similarity**
- **I use Index Flat to search vector**

Search Vector Image

❖ Detail Implement

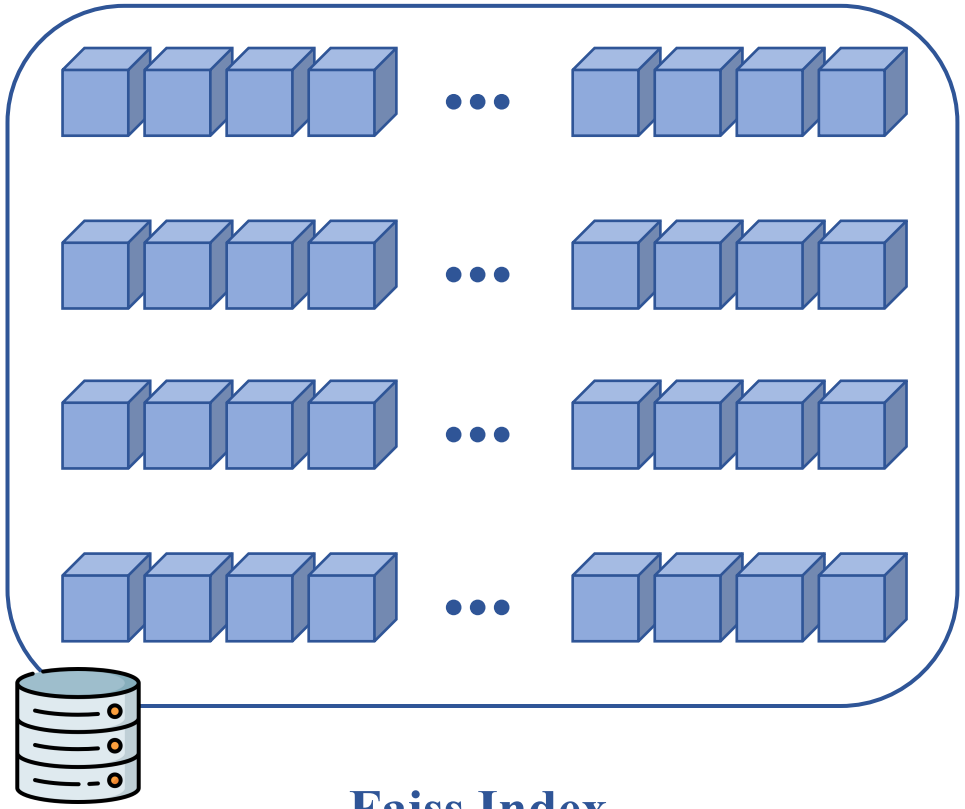
```python
VECTOR_DIM = 300 * 300 * 3

index = faiss.IndexFlatL2(VECTOR_DIM)
label_map = []

for idx, row in df.iterrows():
    image_path = row['image_path']
    label = row['label']

    try:
        vector = image_to_vector(image_path)
        # Add to Faiss index
        index.add(np.array([vector]))
        label_map.append(label)
    except Exception as e:
        print(f"Error processing {image_path}: {e}")

# Save the index and label map
faiss.write_index(index, "employee_images.index")
np.save("label_map.npy", np.array(label_map))
```



Search Vector Image

❖ Searching (Stored Image)

IndexFlatL2



Faiss Index

Query

```
index.search(np.array([query_vector]), k)
```

Query Image

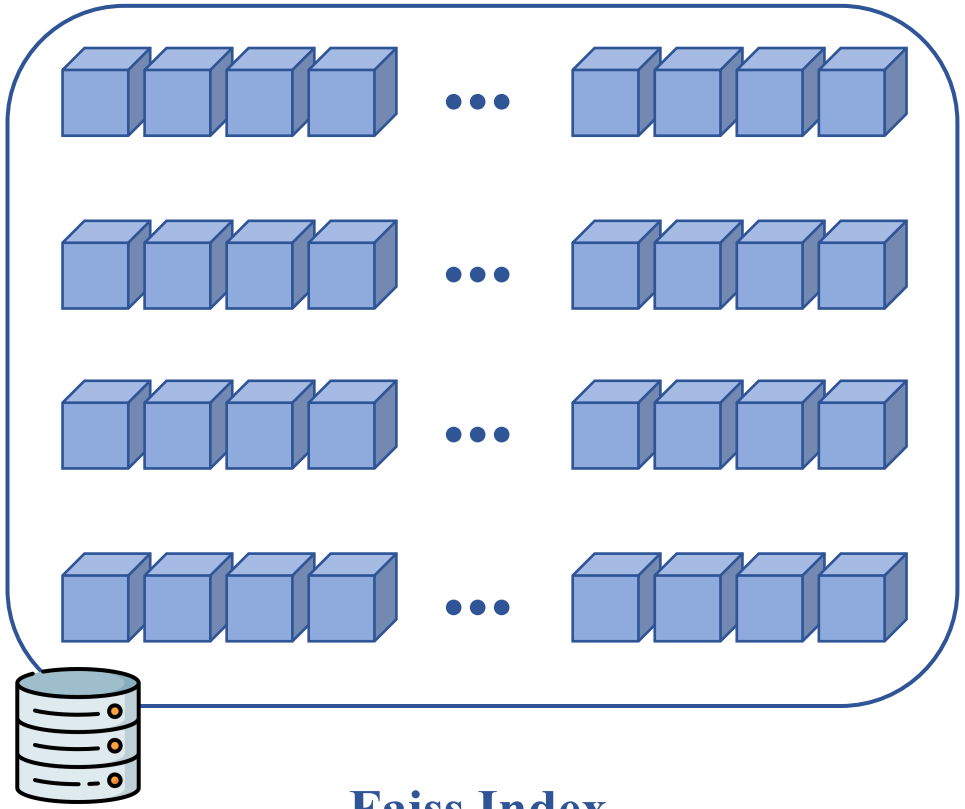
Top K (K = 5)

Search Vector Image

❖ **Searching (New Image)**

Problem?

IndexFlatL2

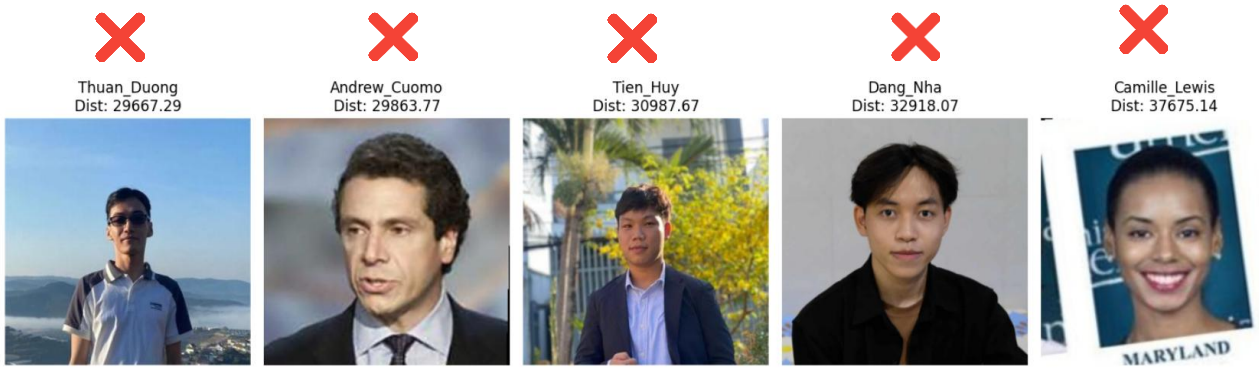


Faiss Index

Query

```
index.search(np.array([query_vector]), k)
```

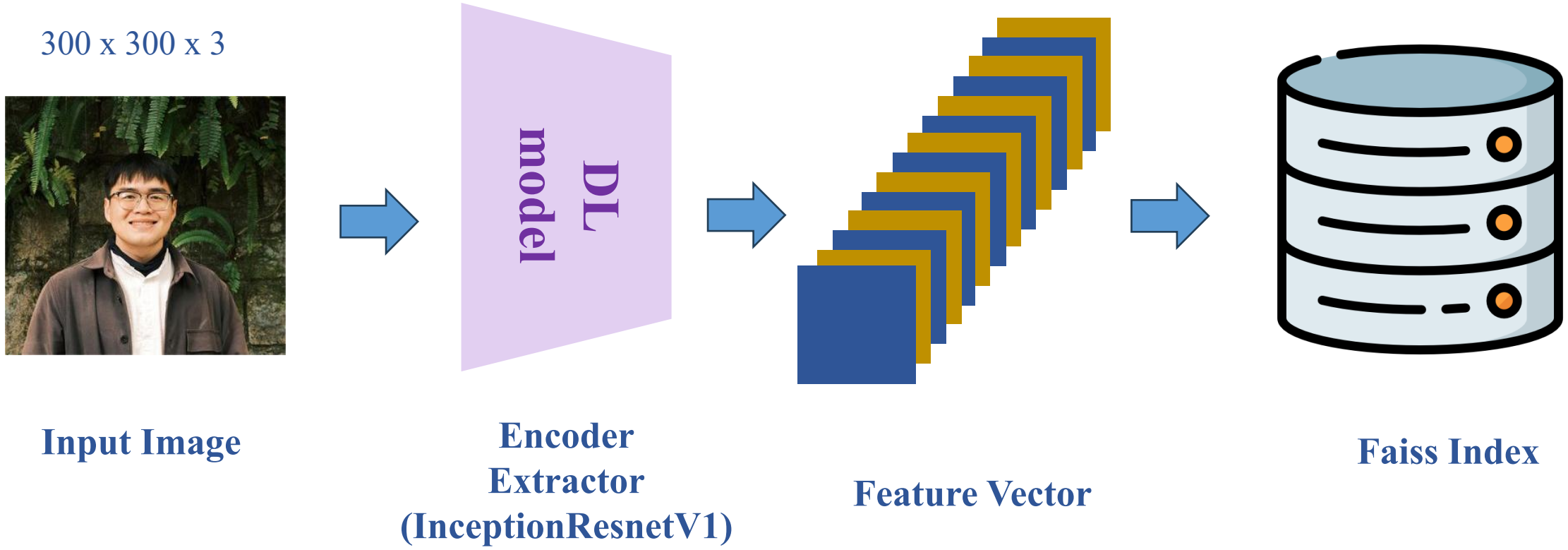
Query Image



Top K (K = 5)

Search Feature Map

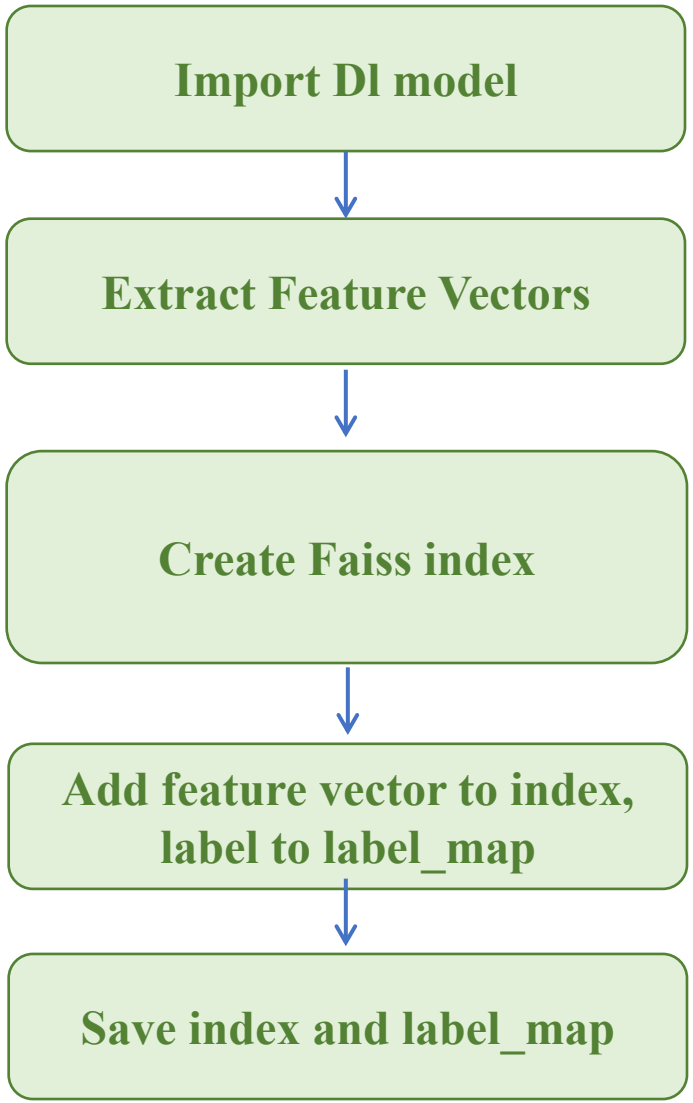
❖ **Extract Feature by using Deep Learning model**



Search Feature Map

❖ Detail Implement

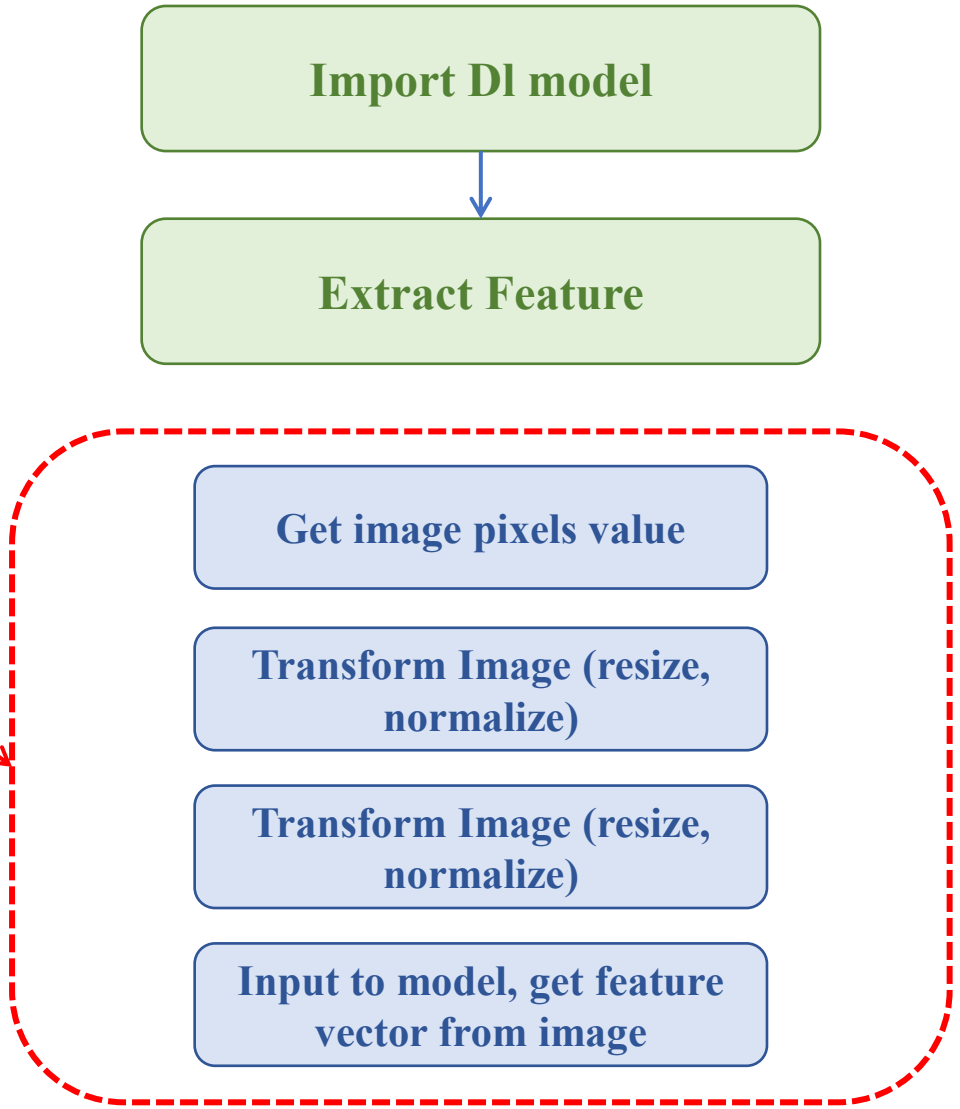
```python
# import Models
from facenet_pytorch import InceptionResnetV1
face_recognition_model = InceptionResnetV1(pretrained='vggface2').eval()


def extract_feature(image_path, model):
    """Extract features from an image using a given model."""
    img = Image.open(image_path).convert('RGB')
    img_tensor = transform(img).unsqueeze(0)
    with torch.no_grad():
        features = model(img_tensor)
    return features.squeeze().numpy()

# Create Faiss index
VECTOR_DIM = 512
index = faiss.IndexFlatIP(VECTOR_DIM)
label_map = []

# Add feature vector to index
for idx, row in tqdm(df.iterrows(), total=len(df)):
    features = extract_feature(row['image_path'], face_recognition_model)
    index.add(np.array([features]))
    label_map.append(row['label'])
```



Search Feature Map

❖ Detail Implement

```python
# import Models
from facenet_pytorch import InceptionResnetV1
face_recognition_model = InceptionResnetV1(pretrained='vggface2').eval()

def extract_feature(image_path, model):
    """Extract features from an image using a given model."""
    img = Image.open(image_path).convert('RGB')
    img_tensor = transform(img).unsqueeze(0)
    with torch.no_grad():
        features = model(img_tensor)
    return features.squeeze().numpy()

# Create Faiss index
VECTOR_DIM = 512
index = faiss.IndexFlatIP(VECTOR_DIM)
label_map = []

# Add feature vector to index
for idx, row in tqdm(df.iterrows(), total=len(df)):
    features = extract_feature(row['image_path'], face_recognition_model)
    index.add(np.array([features]))
    label_map.append(row['label'])
```

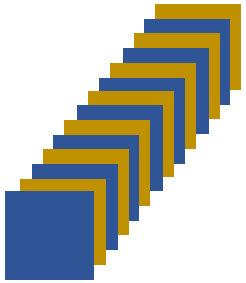


Search Feature Map

❖ **Searching (Stored Image)**

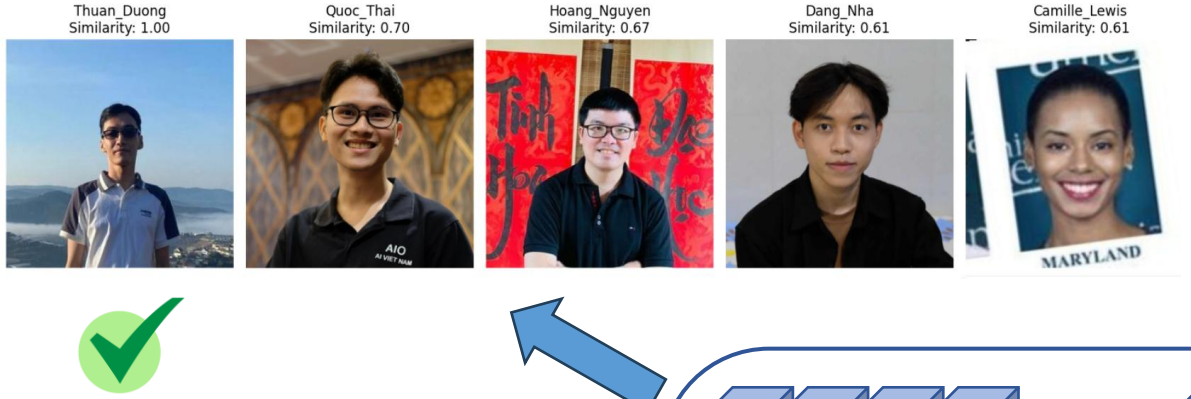
Query Image

DL model

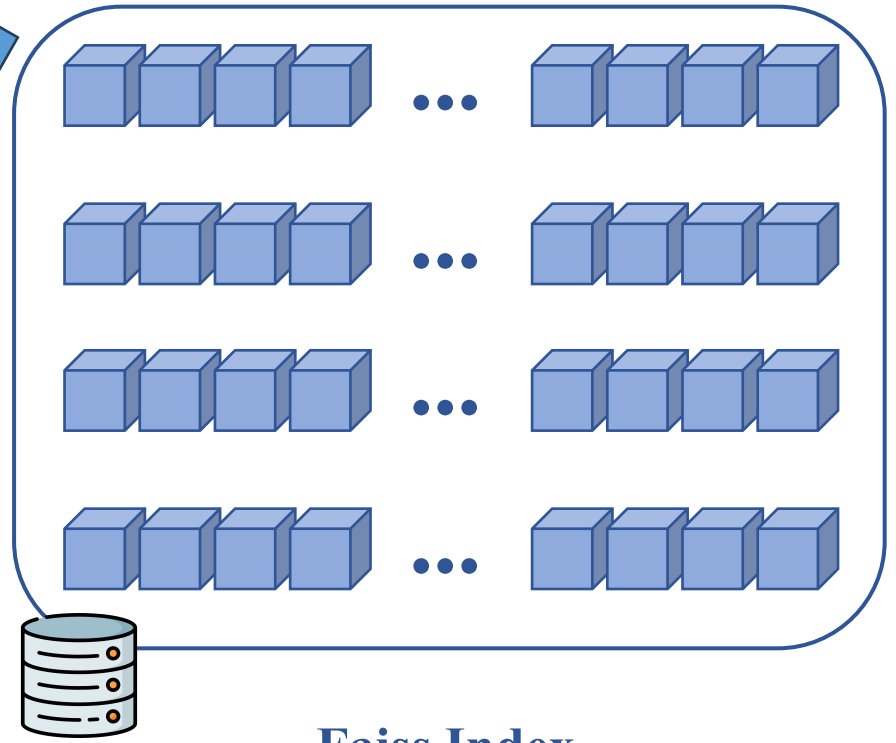


Feature Vector

Query



IndexFlatIP



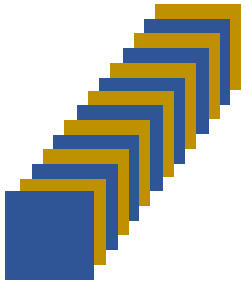
Faiss Index

Search Feature Map

❖ Searching (New Image)

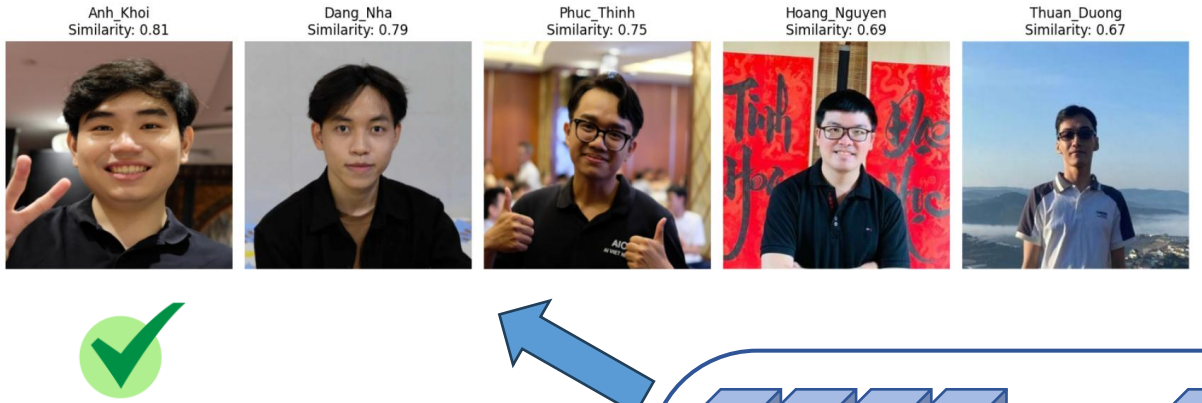
Query Image

DL model

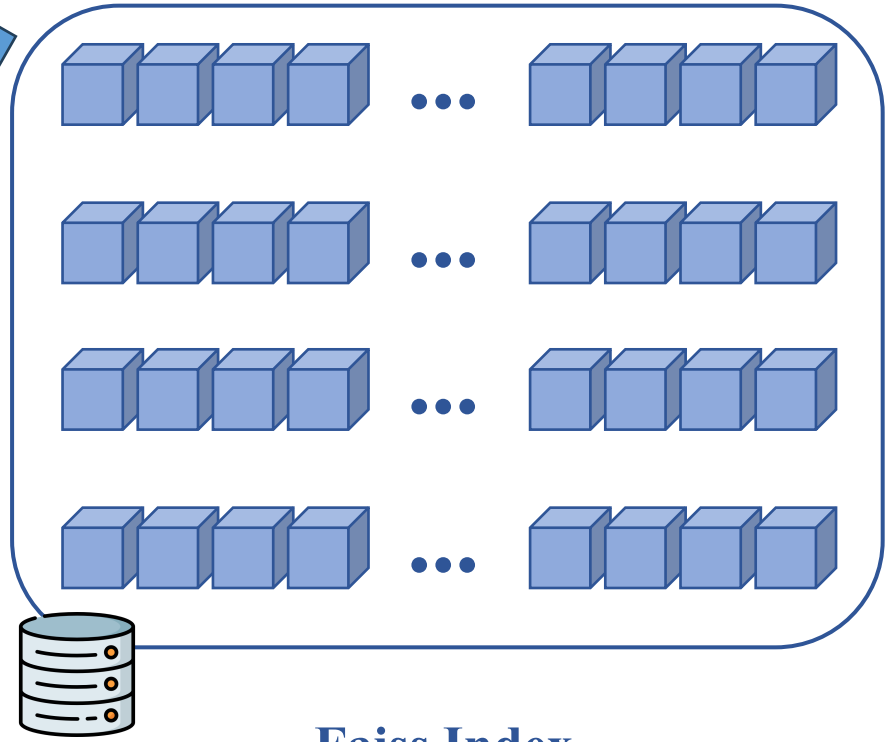


Feature Vector

Query



IndexFlatIP



Faiss Index

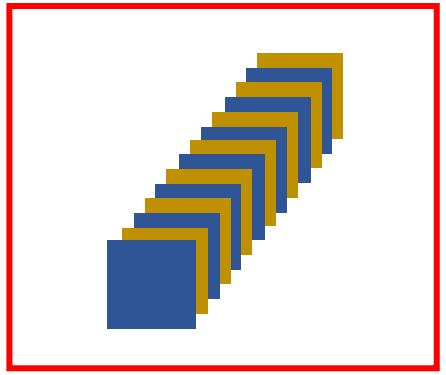
Search Feature Map

❖ **Searching**

Query Image

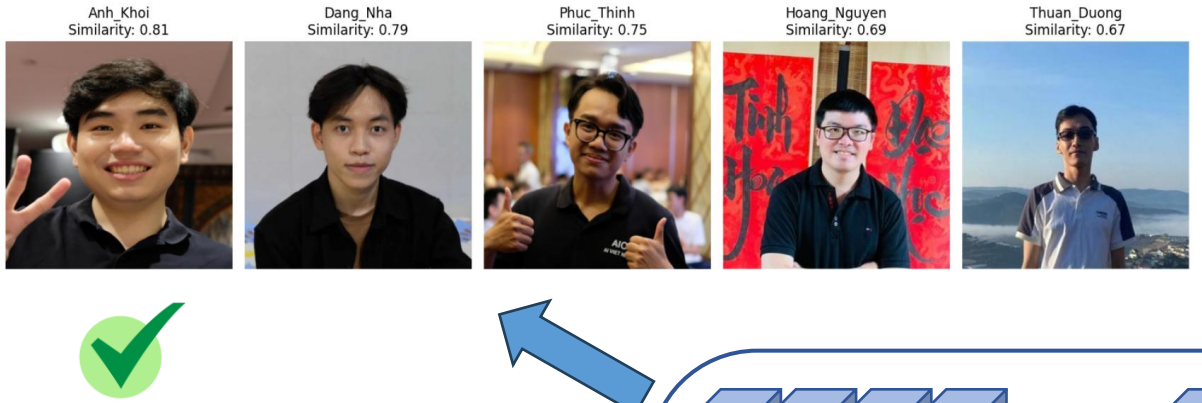
Better Result come from quality of stored vector

DL model

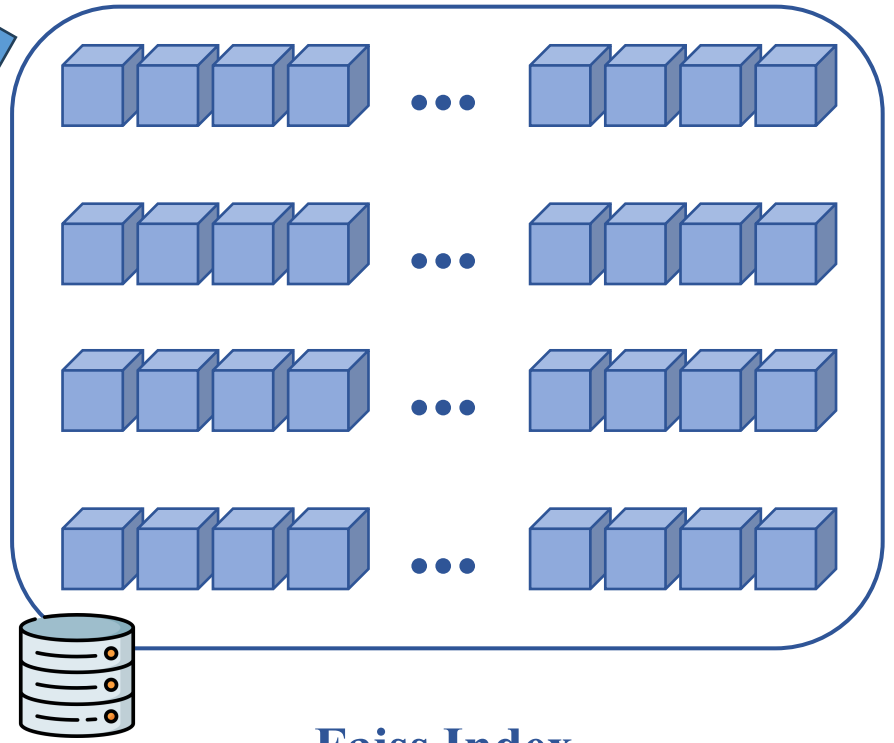


Feature Vector

Query



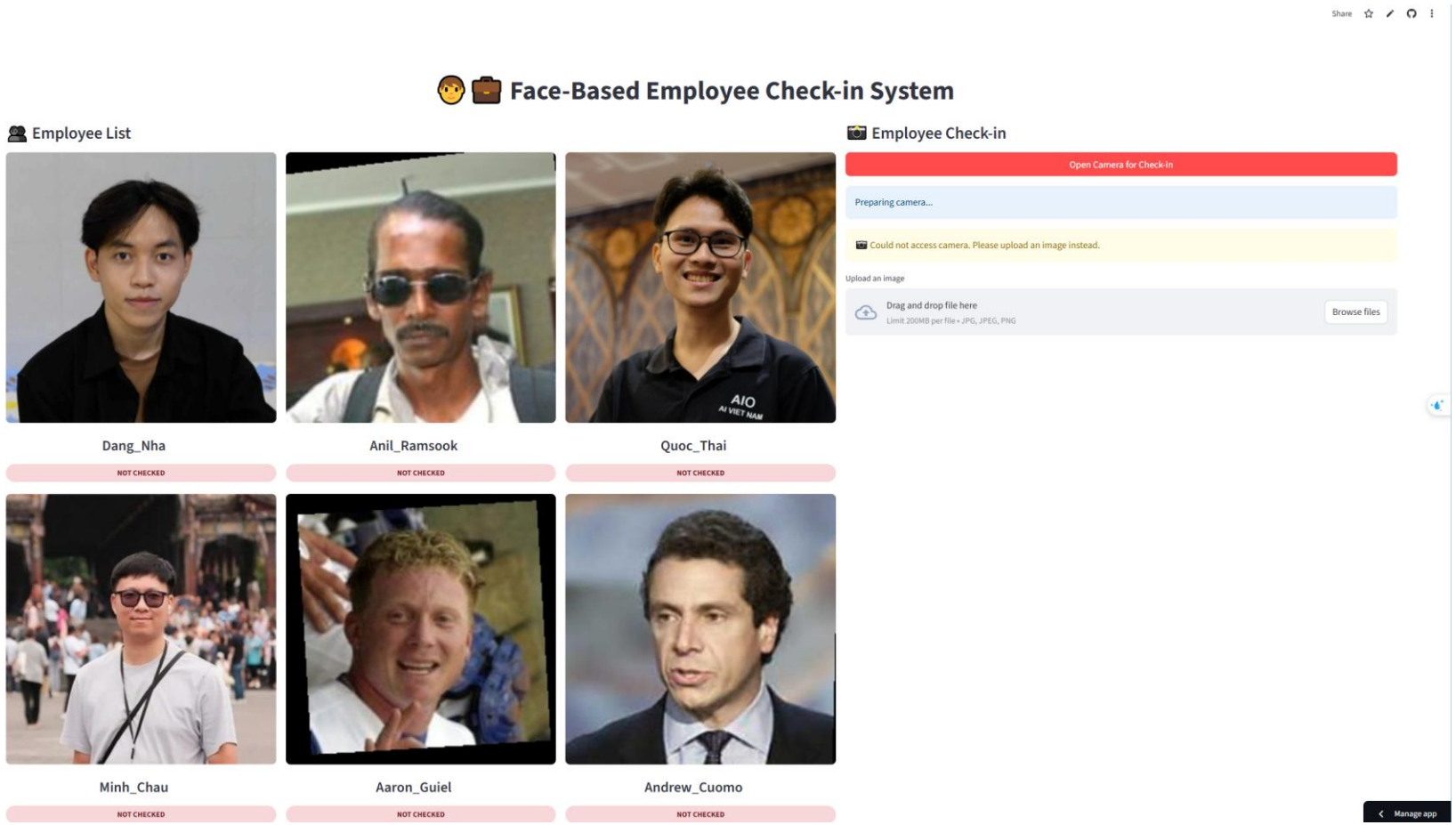
IndexFlatIP



Faiss Index

Project: Check Attendance

❖ Deployment



[Link GitHub Repository]

[Link Demo (Camera -> Uploaed Img)]


Thank you!